Simple employee program

```
class Employee {

    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {

        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    void display() {

        System.out.println("Id     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Salary : " + salary);
    }
}

public class EmployeeMain {

    public static void main(String[] args) {

        Employee employee1 = new Employee(101, "John",
45000.0);
        Employee employee2 = new Employee(102, "Rahul",
50000.0);
        Employee employee3 = new Employee(103, "Kavin",
55000.0);
        Employee employee4 = new Employee(104, "Anu",
48000.0);
        Employee employee5 = new Employee(105, "Vijay",
60000.0);

        employee1.display();
        System.out.println();

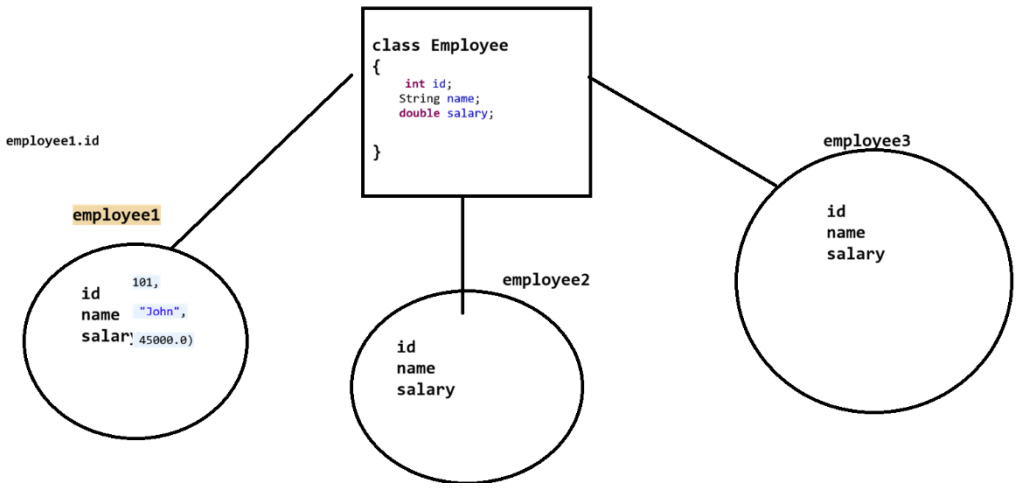
        employee2.display();
```

```
        System.out.println();

        employee3.display();
        System.out.println();

        employee4.display();
        System.out.println();

        employee5.display();
    }
}
```



Simple inheritance program

```
class Animal
{
    public String name;
    public int age;

    public void eat()
    {
        System.out.println(name + " is eating.");
    }
```

```
    public void display()
    {
        System.out.println("Name : " + name);
        System.out.println("Age  : " + age);
    }
}

class Dog extends Animal
{
     String breed;

}

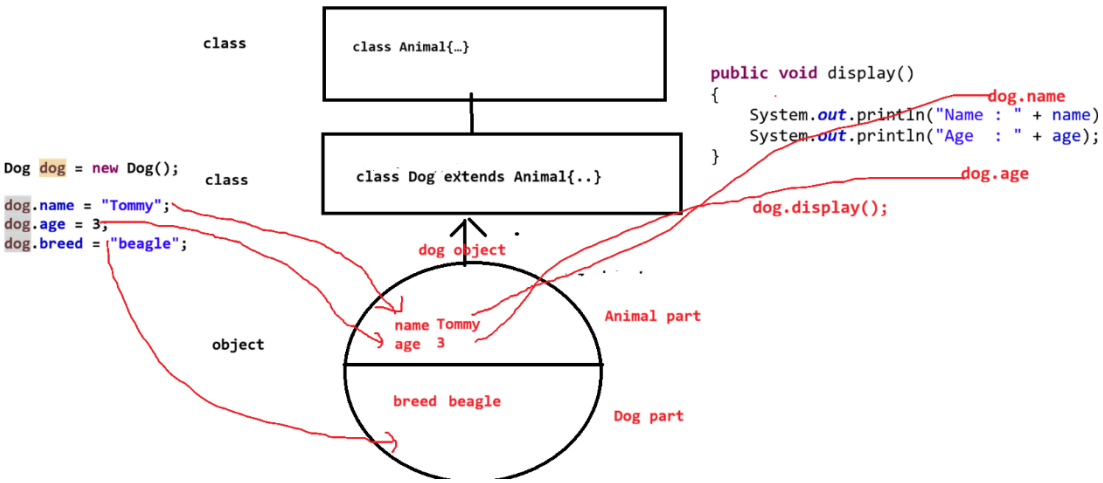


public class AnimalMain
{
    public static void main(String[] args)
    {
     System.out.println("----- Creating Dog object -----
");
        Dog dog = new Dog();

        dog.name = "Tommy";
        dog.age = 3;
        dog.breed = "beagle";


        dog.display();
        dog.eat();

    }
}
```



//Example 1

```
class Animal
{
    void sound()
   {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal
{
   //sub class overridden method
         @Override
   void sound()
   {
      System.out.println("Dog barks");
   }
}

public class MethodOverridingMain
{
   public static void main(String[] args)
   {
```

```
        Dog dog = new Dog();
        dog.sound();


    }
}
```

```
//Example 2- static method - inherited / not

class Animal
{
    static void eat()
    {
        System.out.println("Static Method from Animal");
    }

    void sound()
    {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal
{


}

public class MethodOverridingMain
{
    public static void main(String[] args)
    {

       // Animal.eat();
```

```
        Dog.eat();

        System.out.println();


    }
}


//Example 3

class Animal
{
    static void eat()
    {
        System.out.println("Static Method from Animal");
    }

}

class Dog extends Animal
{
  // @Override
    static void eat()
    {
        System.out.println("Static Method from Dog");
    }

}

public class MethodOverridingMain
{
    public static void main(String[] args)
    {

       // Animal.eat();
        Dog.eat();
```

```
    }
}

//Example 4

class Animal
{
    static void display()
    {
        System.out.println("Static Method from Animal");
    }


}

class Dog extends Animal
{
          //@Override
    static void display()
    {
        System.out.println("Static Method from Dog");
    }

    //Method Overriding

}

public class MethodOverridingMain
{
    public static void main(String[] args)
    {

        Animal.display();
        Dog.display();
```

```
  }
}
```

Variable hiding

```
class Animal {

    int age = 5;
    String name = "Tom";
}

class Dog extends Animal {

    float age = 10.5f; //same dm name , but type is
different
    String name = "Max";//same dm name + same type

    void display() {

        System.out.println("Dog age       : " + age);
        System.out.println("Dog name       : " + name);


    }
}

public class AnimalMain {

    public static void main(String[] args) {

        Dog d = new Dog();
        d.display();
    }
}
```

Using super on data member

```
class Animal {
```

```
    int age = 5;
    String name = "Tom";
}

class Dog extends Animal {

    float age = 10.5f;
    String name = "Max";

    void display() {

        System.out.println("Dog age      : " + age);//10.5
        System.out.println("Animal age   : " +
super.age);//5

        System.out.println("Dog name      : " +
name);//Max

        System.out.println("Animal name      : " +
super.name);//Tom
    }
}

public class AnimalMain {

    public static void main(String[] args) {

        Dog d = new Dog();
        d.display();
    }
}
```

Using super on methods

```
class Animal {

    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
```

```
    @Override
    void sound() {
        System.out.println("Dog barks");
    }

    void display() {

        System.out.println("Calling subclass method:");
        sound();          // Calls Dog's overridden method

        System.out.println();

        System.out.println("Calling superclass method:");
        super.sound();    // Calls Animal's method
    }
}

public class AnimalMain {

    public static void main(String[] args) {

        Dog d = new Dog();
        d.display();
    }
}
```

Constructor overloading

```
class Employee {

    int id;
    String name;
    double salary;

    // user defined Default Constructor
    Employee() {

    }

    // Constructor with one parameter
    Employee(int id) {
```

```
        this.id = id;
    }

    // Constructor with two parameters
    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Constructor with three parameters
    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    void display() {
        System.out.println("Id     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Salary : " + salary);
    }
}

public class EmployeeMain {

    public static void main(String[] args) {

        Employee employee1 = new Employee();
        employee1.display();

        System.out.println();

        Employee employee2 = new Employee(101);
        employee2.display();

        System.out.println();

        Employee employee3 = new Employee(102, "John");
        employee3.display();

        System.out.println();
```

```
        Employee employee4 = new Employee(103, "Rahul",
55000.0);
        employee4.display();
    }
}
```

Constructor calling using this

```
class Employee {

    int id;
    String name;
    double salary;

    Employee() {
        System.out.println("Default Constructor");
    }

    Employee(int id) {
        this.id = id;

    }

    Employee(int id, String name) {
        this.id = id;
        this.name = name;

    }

    Employee(int id, String name, double salary) {
        this(id, name);
         this.salary = salary;

        /*
         this.id = id;
         this.name = name;
         this.salary= salary;

         */
```

```
    }
    void display() {
        System.out.println("Id     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Salary : " + salary);
    }

}

public class EmployeeMain {

    public static void main(String[] args) {

        Employee employee = new Employee(101, "John",
45000.0);
        employee.display();
    }
}
```

Constructor calling using super

```
class Employee {

    int id;
    String name;

    Employee(int id, String name) {

        this.id = id;
        this.name = name;
    }

    void display() {

        System.out.println("Id     : " + id);
        System.out.println("Name   : " + name);
    }
}

class Manager extends Employee {
```

```
    double salary;

    Manager(int id, String name, double salary) {

  // Calls the Employee constructor
        super(id, name);
        this.salary = salary;

    }

    void displayManager() {

        display();   // Calls the Employee class method

        System.out.println("Salary : " + salary);
    }
}

public class EmployeeMain {

    public static void main(String[] args) {

        Manager manager = new Manager(101, "Priya",
50000.0);

        manager.displayManager();
    }
}
```

Inheritance flow

```
class Animal
{
    String name;
    int age;

    //user defined Default Constructor

    Animal()
```

```
    {
        System.out.println("Animal Default Constructor");
    }


    //Parameterized constructor

    Animal(String name, int age)
    {
        System.out.println("Animal Parameterized
Constructor");
        this.name = name;
        this.age = age;
    }


    void displayAnimal()
    {
        System.out.println("Animal Name : " + name);
        System.out.println("Animal Age  : " + age);
    }
}

class Dog extends Animal
{
    String breed;

    //user defined Default Constructor
    Dog()
    {

        System.out.println("Dog Default Constructor");
    }

    // Parameterized Constructor
    Dog(String name, int age, String breed)
    {

      this.name = name;
      this.age = age;
      this.breed = breed;
```

```
      System.out.println("Dog Parameterized Constructor");
    }

    void displayDog()
    {
        displayAnimal();
        System.out.println("Breed       : " + breed);
    }
}

public class AnimalMain
{
    public static void main(String[] args)
    {
     /*
        System.out.println("Dog d1 = new Dog();");
        Dog d1 = new Dog();
        */


        Dog d2 = new Dog("Tommy", 3, "Labrador");

        d2.displayDog();


    }
}
```